

Course Assignment Report

Web Automation Testing using Selenium WebDriver with Python

Scenario 1: Implementation of Selenium Python BDD Framework & Debugging

Name: Oindriyel Mondal

Operating System: Arch Linux (Rolling Release)

Stream: CSE (IoT, Cyber Security & Blockchain Technology) Browser / Engine: Mozilla Firefox / Marionette

Campus: University of Engineering & Management, Kolkata

Driver Binary: GeckoDriver (/usr/bin/geckodriver)

Contents

1	Introduction	1
1.1	Objectives	2
1.2	Tools & Technology Stack	2
2	Project Structure & Environment Setup	3
2.1	Arch Linux Native Setup	3
3	Assignment 1: Implementation of Selenium BDD Scenario	3
3.1	Objective	3
3.2	Target Application	3
3.3	Key Implementation Architecture	3
3.4	Source Code Listing	5
4	Execution Output & Result Verification	8
4.1	Console Execution Log	8
4.2	Step Verification Breakdown	8
4.3	Debugging & Diagnostic Artifact Handling	8
5	Conclusion & Key Learning Outcomes	9

1. Introduction

This report documents the design, architectural configuration, and successful execution of Assignment 1: Implementation of Selenium Python under the automation curriculum. The core

focus centers on establishing a Behavior-Driven Development (BDD) style test framework directly in Python, replacing brittle fixed delays with condition-based explicit synchronization, configuring native browser drivers under Linux, and implementing automated triage routines to capture diagnostic artifacts during execution failures.

1.1 Objectives

- Establish a robust, self-contained Python BDD automation class modeling standard Gherkin syntax step primitives (**GIVEN**, **WHEN**, **THEN**, **AND**).
- Configure automated browser instantiation for native Firefox execution utilizing system-managed **geckodriver** under Arch Linux.
- Implement condition-based synchronization using Selenium's **WebDriverWait** and **expected_conditions** to completely eliminate arbitrary delays (**time.sleep**).
- Verify web UI controls, page metadata, header text, and navigation transitions against public benchmark web applications.
- Build automated failure recovery hooks capable of capturing full-page viewport screenshots into an isolated diagnostics directory whenever an assertion or driver timeout occurs.

1.2 Tools & Technology Stack

Category	Tool / Library	Operational Purpose
Language	Python 3.12+	Core scripting and execution engine
Automation API	Selenium WebDriver 4.x	Browser orchestration and DOM element interaction
Browser Driver	GeckoDriver	Official W3C WebDriver implementation for Firefox (/usr/bin/geckodriver)
Browser Engine	Mozilla Firefox	Target browser for functional testing and UI execution
Operating System	Arch Linux	Native Linux execution host without reliance on bundled Chrome managers
IDE / Environment	PyCharm / Virtualenv	Isolated Python development runtime environment
Assertions	Native Assertions	Step-level validation and contract verification

Table 1: System Environment and Tooling Configuration

2. Project Structure & Environment Setup

To ensure maintainability, cleanliness, and ease of deployment, the assignment is structured as an isolated, self-contained test executable. Dynamic environment variables allow switching between headed and headless execution modes seamlessly.

```
assignment1_suite/  
  .venv/                                # Python virtual environment (isolated  
    site-packages)  
  Assignment1.py                        # Self-contained BDD scenario and execution harness  
  debug_screenshots/                   # Timestamped failure artifacts directory  
    (auto-created)  
    assignment1_failure_<timestamp>.png
```

2.1 Arch Linux Native Setup

The environment relies on native Arch package repositories rather than external dynamic driver managers, ensuring consistent system library linking:

```
# System dependencies installation via pacman  
sudo pacman -S --needed firefox geckodriver python  
  
# Virtual environment provisioning and dependency installation  
python -m venv .venv  
source .venv/bin/activate  
pip install selenium
```

3. Assignment 1: Implementation of Selenium BDD Scenario

3.1 Objective

Construct an end-to-end BDD test scenario targeting <https://the-internet.herokuapp.com>. The script must navigate to the portal, validate the document title and primary header element (<h1>), execute an element click event, and verify the resulting URL route using explicit condition waits.

3.2 Target Application

- Target Base URL: <https://the-internet.herokuapp.com>
- Target Sub-route: <https://the-internet.herokuapp.com/checkboxes>
- Element Locators Employed: By.TAG_NAME (<h1>) and By.LINK_TEXT ("Checkboxes").

3.3 Key Implementation Architecture

The implementation separates driver management, step abstractions, and scenario execution into clean logical layers:

- **get_firefox_driver()**: Discovers and binds the local GeckoDriver binary via path inspection (`/usr/bin/geckodriver` or `shutil.which`), initializes the Firefox options container (supporting headless toggles and 1920×1080 viewport sizing), and enforces a baseline implicit timeout.
- **Assignment1BDDScenario**: Implements structured BDD methods (`given_i_open_url`, `when_page_title_`, `then_page_header_equals`, `and_i_click_link_by_text`, `then_current_url_contains`) wrapping standard Selenium calls with strict explicit assertions.
- **run_scenario()**: Manages test lifecycle execution, timing, exception logging, and automatic screenshot triage.

3.4 Source Code Listing

The following listing demonstrates the complete implementation of the scenario:

```
1 import os
2 import sys
3 import time
4 import shutil
5 import logging
6 from selenium import webdriver
7 from selenium.webdriver.common.by import By
8 from selenium.webdriver.firefox.service import Service
9 from selenium.webdriver.firefox.options import Options
10 from selenium.webdriver.support.ui import WebDriverWait
11 from selenium.webdriver.support import expected_conditions as EC
12 from selenium.common.exceptions import WebDriverException, TimeoutException
13
14 logging.basicConfig(
15     level=logging.INFO,
16     format="%(asctime)s [%(levelname)s] %(message)s",
17     datefmt="%H:%M:%S"
18 )
19 logger = logging.getLogger("BDD_Assignment1")
20
21 def get_firefox_driver(headless: bool = False) -> webdriver.Firefox:
22     gecko_path = os.getenv("GECKODRIVER_PATH", "/usr/bin/geckodriver")
23
24     if not os.path.exists(gecko_path):
25         gecko_path = shutil.which("geckodriver")
26
27     if not gecko_path:
28         logger.error("geckodriver not found! Install via: sudo pacman -S
29             geckodriver")
30         sys.exit(1)
31
32     logger.info(f"Using geckodriver at: {gecko_path}")
33
34     firefox_options = Options()
35     if headless or os.getenv("HEADLESS", "0") == "1":
36         logger.info("Running in headless mode...")
37         firefox_options.add_argument("--headless")
38
39     firefox_options.add_argument("--width=1920")
40     firefox_options.add_argument("--height=1080")
41
42     service = Service(executable_path=gecko_path)
43     driver = webdriver.Firefox(service=service, options=firefox_options)
44     driver.implicitly_wait(5)
45     return driver
46
47 class Assignment1BDDScenario:
48     def __init__(self, driver: webdriver.Firefox, timeout: int = 10):
49         self.driver = driver
50         self.wait = WebDriverWait(driver, timeout)
51
52     def given_i_open_url(self, url: str):
53         logger.info(f"[GIVEN] Navigating to URL: {url}")
```

```
53     self.driver.get(url)
54
55     def when_page_title_contains(self, expected_title_substring: str):
56         logger.info(f"[WHEN] Verifying page title contains:
57             '{expected_title_substring}'")
58         self.wait.until(EC.title_contains(expected_title_substring))
59         actual_title = self.driver.title
60         assert expected_title_substring in actual_title, (
61             f"Expected '{expected_title_substring}' in title, got:
62             '{actual_title}'")
63
64         logger.info(f"        Page title verified: '{actual_title}'")
65
66     def then_page_header_equals(self, expected_header: str):
67         logger.info(f"[THEN] Checking header element text equals:
68             '{expected_header}'")
69         header_el = self.wait.until(
70             EC.visibility_of_element_located((By.TAG_NAME, "h1")))
71
72         assert header_el.text == expected_header, (
73             f"Expected header '{expected_header}', but found '{header_el.text}'")
74
75         logger.info(f"        Header verified: '{header_el.text}'")
76
77     def and_i_click_link_by_text(self, link_text: str):
78         logger.info(f"[AND] Clicking link with text: '{link_text}'")
79         link_el = self.wait.until(
80             EC.element_to_be_clickable((By.LINK_TEXT, link_text)))
81
82         link_el.click()
83
84     def then_current_url_contains(self, expected_slug: str):
85         logger.info(f"[THEN] Verifying current URL contains: '{expected_slug}'")
86         self.wait.until(EC.url_contains(expected_slug))
87         current_url = self.driver.current_url
88         assert expected_slug in current_url, (
89             f"Expected '{expected_slug}' in current URL: '{current_url}'")
90
91         logger.info(f"        URL verified: '{current_url}'")
92
93     def run_scenario():
94         driver = None
95         target_url = "https://the-internet.herokuapp.com"
96         screenshot_dir = "debug_screenshots"
97
98         try:
99             driver = get_firefox_driver(headless=False)
100             scenario = Assignment1BDDScenario(driver=driver, timeout=10)
101
102             logger.info("=====")
103             logger.info("Starting Assignment 1: Selenium BDD Scenario")
104             logger.info("=====")
105
106             # Execute Scenario Steps
107             scenario.given_i_open_url(target_url)
108             scenario.when_page_title_contains("The Internet")
```

```
106     scenario.then_page_header_equals("Welcome to the-internet")
107     scenario.and_i_click_link_by_text("Checkboxes")
108     scenario.then_current_url_contains("/checkboxes")
109
110     logger.info("=====")
111     logger.info("Scenario PASSED: All BDD steps executed cleanly!")
112     logger.info("=====")
113
114     except (AssertionError, TimeoutException, WebDriverException) as err:
115         logger.error(f"[FAILURE] Test failed: {err}")
116         if driver:
117             os.makedirs(screenshot_dir, exist_ok=True)
118             screenshot_file = os.path.join(
119                 screenshot_dir, f"assignment1_failure_{int(time.time())}.png"
120             )
121             driver.save_screenshot(screenshot_file)
122             logger.error(f"[DEBUG] Failure screenshot captured at:
123                         {screenshot_file}")
124             sys.exit(1)
125
126     finally:
127         if driver:
128             logger.info("Tearing down Firefox session...")
129             driver.quit()
130
131 if __name__ == "__main__":
132     run_scenario()
```

4. Execution Output & Result Verification

4.1 Console Execution Log

The test was executed within the activated virtual environment on an Arch Linux workstation. The resulting standard output demonstrates smooth session initialization, sequential step resolution, and clean driver teardown:

```
.venv  python Assignment1.py

23:00:01 [INFO] Using geckodriver at: /usr/bin/geckodriver
23:00:04 [INFO] =====
23:00:04 [INFO] Starting Assignment 1: Selenium BDD Scenario
23:00:04 [INFO] =====
23:00:04 [INFO] [GIVEN] Navigating to URL: https://the-internet.herokuapp.com
23:00:10 [INFO] [WHEN] Verifying page title contains: 'The Internet'
23:00:10 [INFO]         Page title verified: 'The Internet'
23:00:10 [INFO] [THEN] Checking header element text equals: 'Welcome to
the-internet'
23:00:10 [INFO]         Header verified: 'Welcome to the-internet'
23:00:10 [INFO] [AND] Clicking link with text: 'Checkboxes'
23:00:10 [INFO] [THEN] Verifying current URL contains: '/checkboxes'
23:00:10 [INFO]         URL verified:
'https://the-internet.herokuapp.com/checkboxes'
23:00:10 [INFO] =====
23:00:10 [INFO] Scenario PASSED: All BDD steps executed cleanly!
23:00:10 [INFO] =====
23:00:10 [INFO] Tearing down Firefox session...
```

4.2 Step Verification Breakdown

BDD Step	Condition Verified	Expected Assertion	Result
GIVEN	Base Page Loading	Network response code 200 via driver load	PASS
WHEN	Title Synchronization	Page title contains substring "The Internet"	PASS
THEN	Header Text Presence	<h1> DOM text matches "Welcome to the-internet"	PASS
AND	Clickable Link Element	Element with text "Checkboxes" is clickable	PASS
THEN	Route Transition	Current address contains route "/checkboxes"	PASS

Table 2: Detailed Step Execution and Verification Results

4.3 Debugging & Diagnostic Artifact Handling

The execution harness implements an active exception catcher listening for `AssertionError`, `TimeoutException`, and generalized `WebDriverException`:

1. When a failure occurs, execution jumps immediately to the exception handler without suppressing traceback context.
2. The framework checks for an active `driver` session, creates the directory `debug_screenshots/` if missing, and dumps a high-resolution viewport capture named `assignment1_failure_<unix_timestamp>`

3. Session disposal is guaranteed via the `finally` block, which invokes `driver.quit()` to prevent orphaned GeckoDriver background daemons or zombie Firefox processes.

5. Conclusion & Key Learning Outcomes

The implementation successfully fulfills all objectives stipulated for Assignment 1:

- Native Linux Driver Optimization: Configured direct execution through Arch Linux's official GeckoDriver binary, bypassing the overhead and instability of external automatic driver downloaders.
- Dynamic Synchronization: Completely avoided race conditions by pairing `WebDriverWait` with explicit conditions (`EC.title_contains`, `EC.visibility_of_element_located`, `EC.element_to_be_clickable`, `EC.url_contains`).
- BDD Abstraction: Established readable, scalable testing semantics modeling Gherkin syntax step definitions directly within an object-oriented Python structure.
- Automated Diagnostics: Built an enterprise-grade failure-handling mechanism ensuring automated screenshot artifact logging for immediate post-mortem debugging.